СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 3

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 4

ВВЕДЕНИЕ 5

ГЛАВА 1 7

1.1 Анализ существующих курсов по машинному обучению 7

1.1.1 Машинное обучение на ФКН НИУ ВШЭ 7

1.1.2 Машинное обучение на ВМК МГУ 7

1.1.3 Открытый курс по машинному обучению mlcourse.ai ... 8

1.1.4 Deep Learning School МФТИ 8

1.1.5 Курс машинного обучения girafe-ai 8

1.1.6 Karpov.Courses: Инженер машинного обучения 9

1.1.7 Обобщение результатов анализа 9

1.2 Анализ инструментов для автоматической проверки 10

1.2.1 nbgrader 11

1.2.2 Otter-Grader 11

1.2.3 CodeGrade 11

1.2.4 PyEvalAI 11

1.2.5 Яндекс.Контест 12

1.2.6 Stepik 12

1.2.7 GitHub Classroom 12

1.2.8 Kaggle InClass 13

1.2.9 Сравнительный анализ 13

1.3 Итоги анализа 14

1.4 Формулировка гипотезы и образ решения 14

1.4.1 Формулировка гипотезы 14

1.4.2 Образ решения 15

ГЛАВА 2 17

2.1 Рабочая программа дисциплины 17

2.1.1 Пререквизиты 17
2.1.2 Разделы 17
2.1.3 Темы 17
2.1.4 Оцениваемые задания 18
2.1.5 Результаты обучения 20
2.2 Внедрение автоматизированной проверки в nbgrader 20
2.2.1 Требования к работе 21
2.2.2 Реализация расширения nbgrader 22
2.2.3 Создание задания преподавателем 22
2.2.4 Генерация студенческой версии 23
2.2.5 Конфигурация и запуск проверки 23
2.3 Создание системы для проверки проектов 24

ГЛАВА 3 25

ЗАКЛЮЧЕНИЕ 26

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 26

А РУКОВОДСТВО ПО ИСПОЛЬЗОВАНИЮ ШАБЛОНА 29

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

БЯМ — большая языковая модель

ODS — Open Data Science

NLP — Natural Language Processing

CV — Computer Vision

ИИ — Искусственный Интеллект

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Большая языковая модель (БЯМ) — нейросетевая модель с большим числом параметров, обученная на текстовых данных для генерации и понимания естественно языка

ВВЕДЕНИЕ

Машинное обучение за последнее десятилетие стало одной из ключевых компетенций для IT-специалистов. Спрос на образовательные программы по данному направлению неуклонно растет: курсы предлагают ведущие университеты (НИУ ВШЭ, МГУ, МФТИ), корпоративные школы (ШАД Яндекса, Т-Образование) и открытые платформы (ODS, Stepik, Karpov.Courses). Вместе с ростом числа слушателей обостряется проблема масштабируемости оценивания: преподаватели тратят значительную часть времени на проверку работ, а при увеличении потока студентов качество обратной связи снижается.

Анализ существующих курсов по машинному обучению показал, что автоматизация оценивания остается фрагментарной. Существующие инструменты - юнит-тесты в Jupyter Notebooks, контестные системы, тесты с выбором ответа - эффективны для проверки кода и формализованных ответов, но не позволяют оценивать задания открытого типа: теоретические вопросы, развернутые объяснения, интерпретацию результатов. Такие задания по-прежнему проверяются вручную, что создает узкое место в образовательном процессе. Появление больших языковых моделей (БЯМ) открывает возможность автоматизировать проверку заданий, требующих понимания естественного языка и оценки содержательной корректности ответов.

Новизна работы заключается в разработке и апробации формата образовательного курса по машинному обучению, в котором автоматическая оценка открытых ответов реализована с использованием БЯМ, интегрированного в существующий учебный пайплайн nbgrader. В отличие от решений, ориентированных только на проверку кода и закрытых тестов, предложенный подход объединяет новый формат курса, встроенный БЯМ и его применение на реальной аудитории студентов.

Практическая значимость работы определяется внедрением разработанного курса в образовательный процесс НИУ ВШЭ, а также апробацией системы автоматической проверки на реальной аудитории студентов, что позволяет оценить её применимость и эффективность в условиях учебного процесса и

потенциальную переносимость на другие технические дисциплины.

Целью данной работы является разработка автоматизированной системы проверки учебных заданий на основе больших языковых моделей для курса по машинному обучению, ее интеграция в образовательный процесс и апробация.

Для достижения данной цели были поставлены следующие задачи:

1) Провести анализ существующих курсов по машинному обучению и сформировать профиль целевой аудитории.
2) Исследовать подходы к автоматизированной проверке учебных заданий в образовательной среде.
3) Разработать методологию преподавания и структуру учебного курса по машинному обучению.
4) Разработать контрольные задания, критерии оценивания и учебно-методические материалы курса.
5) Реализовать автоматизированную систему проверки учебных заданий с использованием больших языковых моделей.
6) Провести апробацию разработанного курса и системы автоматической проверки в образовательном процессе НИУ ВШЭ и проанализировать полученные результаты.

Критериями успешности решения выступают: сокращение времени проверки учебных заданий, высокая степень согласованности автоматических оценок с экспертными, а также положительная оценка скорости и качества обратной связи со стороны студентов по итогам апробации.

Работа состоит из введения, трех глав и заключения. В первой главе описана предметная область, проведен анализ существующих курсов и подходов к автоматизированной проверке, сформулированы требования к решению. Во второй главе представлена архитектура разработанного курса и системы автоматической проверки, описаны методология, структура заданий и механизм оценивания на основе БЯМ. В третьей главе приведены результаты апробации курса, анализ метрик качества автоматической проверки и обратная связь от студентов.

ГЛАВА 1

1.1 Анализ существующих курсов по машинному обучению

Для формирования методологической и содержательной основы разрабатываемого курса был проведен анализ X существующих образовательных программ по машинному обучению. Выборка охватывает академические курсы ведущих российских и зарубежных университетов, корпоративные школы, коммерческие онлайн-платформы и открытые образовательные инициативы. Для каждого курса рассматривались формат обучения, содержание программы, система контроля знаний и степень автоматизации процесса оценивания.

1.1.1 Машинное обучение на ФКН НИУ ВШЭ

Курс Е. А. Соколова [1] является одним из наиболее известных академических курсов по машинному обучению. Обучение проходит в очном формате для бакалавров факультета компьютерных наук НИУ ВШЭ, при этом все материалы, включая конспекты лекций и видеозаписи, опубликованы в открытом доступе. Программа охватывает классические разделы машинного обучения. На семинарах студенты реализуют изучаемые алгоритмы в среде Jupyter Notebook и выполняют короткие самостоятельные работы.

Система контроля включает теоретические и практические домашние задания, контрольную работу и экзамен. Теоретические домашние задания включают открытые вопросы с объяснением, а практические подразумевают работу в Jupyter Notebook. Домашние задание принимаются через платформу Anytask.

1.1.2 Машинное обучение на ВМК МГУ

Курс машинного обучения для студентов третьего курса факультета вычислительной математики и кибернетики МГУ [2] по содержанию близок к программе НИУ ВШЭ, но дополнительно включает введение в нейронные сети и алгоритм обратного распространения ошибки. Обучение организовано в очном формате и сопровождается практическими и теоретическими видами контроля. В практические входит работа в

Jupyter Notebook и соревнования на платформе Kaggle, а в теоретические - коллоквиумы, тесты на лекциях и экзамен.
Проверка практических заданий автоматизирована при помощи встроенных тестов в .ipynb-файлах, при этом некоторые задания требуют кросс-проверки между студентами в количестве трех человек. [3]

1.1.3 Открытый курс по машинному обучению mlcourse.ai

Курс сообщества Open Data Science [4] представляет собой полностью онлайн-формат обучения и распространяется бесплатно без отбора слушателей. Темы включают линейные модели, решающие деревья и дополнительные главы по машинному обучению. Длительность курса составляет два месяца.
Проверочные контроли состоят из тестов, дополнительно дается возможность пройти соревнование. Проверка заданий полностью автоматизирована на собственной платформе ODS.ai.

1.1.4 Deep Learning School МФТИ

Школа глубокого обучения (DLS) при МФТИ [5] является бесплатным онлайн-курсом, реализованным на платформе Stepik. Программа разделена на четыре части. В первой части обсуждаются основы машинного обучения и нейронных сетей, во второй части - NLP, третья посвящена CV, а четвертая аудио.
Оценивание включает домашние задания в Jupyter Notebook, тесты, соревнования на платформе Kaggle и итоговый проект. Тесты автоматизированы на платформе Stepik. Домашние задания проверяются в формате кросс-валидации между студентами. Проект проверяется преподавателем, закрепленным за студентом.

1.1.5 Курс машинного обучения girafe-ai

Открытый курс girafe-ai [6] содержит полный цикл изучения машинного обучения: от линейных моделей и метода опорных векторов до нейронных сетей, алгоритма обратного распространения ошибки, эмбеддингов и архитектур seq2seq. Материалы курса и видеозаписи лекций опубликованы в открытом доступе.
Контроль знаний организован через серию заданий и лабораторных работ в Jupyter Notebook, а также итоговый экзамен. Некоторые задания сразу

включают тесты для самопроверки, а некоторые содержат теоретические вопросы.

1.1.6 Karpov.Courses: Инженер машинного обучения

Коммерческий курс [7] реализован полностью в онлайн-формате. Программа ориентирована на подготовку специалистов от базового уровня до внедрения моделей в производственную эксплуатацию: линейные модели, деревья решений, нейронные сети, обработка естественного языка, рекомендательные системы, A/B-тестирование и развертывание моделей. Классические семинары в курсе отсутствуют, практическая подготовка обеспечивается за счет решения прикладных задач на собственной платформе и разработки итогового проекта в формате ML-сервиса с проверкой экспертов.

1.1.7 Обобщение результатов анализа

В таблице 1.1 систематизированы ключевые характеристики рассмотренных курсов. Год соответствует последнему обновлению курса.

Таблица 1.1 — Сравнение курсов по машинному обучению

Курс	**Год**	**Формат**	**Jupyter Notebook**
МО ФКН НИУ ВШЭ	2026	Очный	Да
МО ВМК МГУ	2026	Очный	Да
mlcourse.ai	2026	Онлайн	Да
DLS МФТИ	2026	Онлайн	Да
girafe-ai	2021	Очный	Да
Karpov.Courses	2026	Онлайн	Неизвестно

Продолжение таблицы 1.1

Курс	**Теор. практика**	**Развернутая обратная связь**	**Проек**
МО ФКН НИУ ВШЭ	Да	Нет	Нет
МО ВМК МГУ	Да	Нет	Нет
mlcourse.ai	Нет	Нет	Нет
DLS МФТИ	Да	Да	Stepik, K
girafe-ai	Да	Нет	Нет
Karpov.Courses	Нет	Да, только в проекте	Да

Проведенный анализ шести курсов позволяет зафиксировать следующее:

– 5 из 6 курсов используют практические задания в формате Jupyter Notebook.
– 4 из 6 курсов включают теоретические задания: открытые вопросы, коллоквиумы или экзамен.
– Все 6 курсов ведутся на русском языке; по формату выборка делится поровну: 3 очных и 3 онлайн-курса.
– Итоговый проект предусмотрен только в 2 из 6 курсов - DLS МФТИ и Karpov.Courses; оба реализованы в онлайн-формате и именно по проектам подразумевают развернутую обратную связь.
– Основная программа схожа: линейные модели, решающие деревья, ансамбли и методы оценки качества присутствуют во всех курсах. Различия проявляются в дополнительных темах: академические курсы делают акцент на теоретической базе и математическом обосновании алгоритмов, тогда как коммерческие и прикладные программы расширяют программу за счет тем, связанных с внедрением моделей, обработкой различных типов данных и проектной работой.

Задания открытого типа - теоретические вопросы с развернутым ответом, объяснение результатов, интерпретация моделей - проверяются одним из трех способов: вручную преподавателями, посредством кросс-проверки между студентами, либо исключаются из системы контроля. Каждый подход имеет ограничения: ручная проверка плохо масштабируется, кросс-проверка не гарантирует согласованности оценок, а отказ от заданий открытого типа обедняет образовательный процесс. Таким образом, автоматизация проверки покрывает практические задания и формализованные тесты, но не распространяется на задания, требующие анализа естественного языка. Для выбора подходящего решения необходимо рассмотреть существующие инструменты автоматической проверки.

1.2 Анализ инструментов для автоматической проверки

Задача автоматизации проверки учебных заданий решается различными инструментами: от классических систем юнит-тестирования Jupyter Notebook до коммерческих платформ и исследовательских прототипов на основе больших языковых моделей. Ниже рассмотрены наиболее распространённые решения.

1.2.1 nbgrader

nbgrader — открытая система для создания и проверки заданий в формате Jupyter Notebook, разработанная в UC Berkeley и поддерживаемая сообществом Jupyter [8; 9]. Поддерживает два типа ячеек для проверки: автоматически проверяемые ячейки кода (на основе assert-тестов) и ячейки с открытым ответом. Открытые ответы (теоретические вопросы, текстовые объяснения) проверяются вручную через веб-интерфейс Formgrader. Преподавателю необходимо разметить ячейки в Jupyter, написать тесты и настроить структуру каталогов. Nbgrader доступен как в JupyterHub, так и как консольная утилита.

1.2.2 Otter-Grader

Otter-Grader — открытая система автоматической проверки на Python и R [10; 11]. Как и nbgrader, использует разметку ноутбука специальными комментариями и предоставляет консольную утилиту, однако ключевое отличие состоит в модели исполнения: команда otter assign генерирует из исходного ноутбука студенческую версию и архив для запуска проверки в изолированном Docker-контейнере, что упрощает масштабирование. Открытые вопросы помечаются как manual: true и экспортируются в PDF для ручной проверки во внешней системе. Собственного интерфейса для проверки открытых ответов нет.

1.2.3 CodeGrade

CodeGrade — коммерческая платформа, разработанная в Университете Амстердама [12]. Имеет нативную поддержку Jupyter Notebook, позволяющую визуально настраивать блоки тестов: проверки ввода-вывода, юнит-тесты, линтеры, структурные проверки и пользовательские скрипты. Интегрирован с LMS, включает встроенную проверку на плагиат. У студентов есть доступ к ИИ-ассистенту, но пока этот функционал находится на бета-тестировании.

1.2.4 PyEvalAI

PyEvalAI — исследовательский прототип находящийся в апробации и без открытого кода [13]. Система оценивает как код, так и текстовые ячейки в Jupyter Notebook, комбинируя юнит-тесты с локально развёрнутой открытой БЯМ. Модель получает на вход условие задачи, эталонное решение, ответ

студента и результаты тестов, после чего генерирует численную оценку и персонализированную обратную связь. От преподавателя требуется составить ноутбуки с заданиями и эталонами, а также развернуть сервер с GPU.

1.2.5 Яндекс.Контест

Яндекс.Контест — проприетарная платформа компании Яндекс, реализующая классическую модель онлайн-судьи (stdin/stdout) [14]. Поддерживает программные задачи с тестами и пользовательские проверки. Jupyter Notebook не поддерживается; открытые теоретические вопросы оцениваться не могут. Преподавателю необходимо подготовить условие и тесты.

1.2.6 Stepik

Stepik — образовательная платформа, поддерживающая 18 типов автоматически проверяемых заданий: тесты с выбором ответа, численные задачи, проверка формул через SymPy, регулярные выражения, исполнение кода против тестов, SQL [15]. Для открытых ответов доступна кросс-валидация между студентами или проверка командой курса. Jupyter Notebook не поддерживается — код выполняется во встроенном редакторе. Преподаватель настраивает задания через веб-интерфейс, для нестандартных проверок может написать функцию check(reply) на Python.

1.2.7 GitHub Classroom

GitHub Classroom — бесплатный сервис GitHub для управления учебными заданиями, использующий инфраструктуру Git и GitHub Actions для автоматической проверки [16]. Каждое задание представлено шаблонным репозиторием: при принятии задания студент получает приватную копию, и при каждом `git push` в Linux-окружении GitHub Actions запускаются преподавательские тесты, результаты которых сразу отображаются в интерфейсе репозитория. .

Открытые теоретические вопросы платформа не оценивает — проверка строится исключительно на правилах. Jupyter Notebook нативно не поддерживается, однако ноутбук может быть преобразован в Python-скрипт или проверен через Python библиотеки.

1.2.8 Kaggle InClass

Kaggle InClass — режим закрытых учебных соревнований на платформе Kaggle, позволяющий преподавателю создать соревнование с приватным тестовым набором [17]. Студенты загружают файлы предсказаний, которые автоматически оцениваются по выбранной метрике с публикацией результатов на лидерборде. Проверяется только качество предсказаний — ни код, ни теоретические ответы платформа не анализирует, что требует комбинирования с другими инструментами для полноценного контроля. Преподавателю необходимо подготовить датасет, разметку, ground truth для скрытой части и описание метрики.

1.2.9 Сравнительный анализ

В таблице 1.2 приведено сравнение рассмотренных инструментов по ключевым для настоящей работы критериям.

Таблица 1.2 — Сравнение инструментов автоматической проверки

Инструмент	**Открытые вопросы**	**Jupyter Notebook**
nbgrader	Только вручную	Нативная
Otter-Grader	Только вручную	Нативная
CodeGrade	Только вручную	Нативная
GitHub Classroom	Не поддерживаются	Нет
PyEvalAI	Да, через БЯМ	Нативная
Яндекс.Контест	Не поддерживаются	Нет
Stepik	Только вручную или кросс-валидация	Нет
Kaggle InClass	Не поддерживаются	Нет

– 7 из 8 инструментов либо проверяют открытые вопросы только вручную, либо не поддерживают их вовсе.
– Нативная поддержка Jupyter Notebook реализована у 4 из 8 инструментов; остальные 3 её не имеют.
– Единственный инструмент, сочетающий автоматическую проверку открытых вопросов через БЯМ и нативную поддержку Jupyter Notebook, — PyEvalAI — остаётся исследовательским прототипом с ограниченной апробацией.
– Таким образом, 6 из 7 рассмотренных решений не закрывают потребность в автоматической проверке открытых теоретических вопросов в среде Jupyter Notebook.

1.3 Итоги анализа

Проведённый анализ существующих курсов и инструментов автоматической проверки позволяет сформулировать ключевые наблюдения.

Во-первых, задания открытого типа — теоретические вопросы с развёрнутым ответом, интерпретация результатов, формулировка выводов — присутствуют в большинстве академических курсов по машинному обучению, однако их проверка остаётся либо ручной, либо вынесенной на кросс-валидацию между студентами. Оба подхода плохо масштабируются и не обеспечивают своевременной и согласованной обратной связи.

Во-вторых, среди восьми рассмотренных инструментов автоматической проверки ни один не сочетает нативную поддержку Jupyter Notebook с автоматической оценкой открытых текстовых ответов. Единственное исключение — исследовательский прототип PyEvalAI — не имеет открытого кода и находится на стадии апробации.

В-третьих, наиболее распространённой средой для практических заданий по машинному обучению остаётся Jupyter Notebook: 5 из 6 проанализированных курсов используют именно этот формат. При этом nbgrader — наиболее зрелый инструмент с нативной поддержкой Jupyter Notebook — не предоставляет средств для автоматической оценки открытых ответов, ограничиваясь ручной проверкой через интерфейс Formgrader.

Таким образом, в существующей экосистеме инструментов для обучения машинному обучению сохраняется незакрытая потребность: автоматическая проверка открытых теоретических вопросов и проектных работ в среде Jupyter Notebook. Именно это противоречие определяет направление настоящей работы.

1.4 Формулировка гипотезы и образ решения

1.4.1 Формулировка гипотезы

На основании проведённого анализа формулируется следующая гипотеза: интеграция большой языковой модели в систему проверки заданий на базе nbgrader позволит автоматизировать оценку открытых теоретических вопросов в среде Jupyter Notebook, обеспечив при этом качество и согласованность обратной связи, сопоставимые с ручной проверкой преподавателя, и не требуя от него дополнительных временных затрат

на рутинное оценивание. Помимо этого, применение БЯМ для проверки итоговых проектов позволит обеспечить студентам персонализированную развёрнутую обратную связь по нетиповым открытым заданиям.

Проверка гипотезы предполагает разработку соответствующего программного решения, создание курса по классическому машинному обучению и проведение апробации на реальной учебной аудитории.

1.4.2 Образ решения

Предлагаемое решение состоит из двух взаимосвязанных компонентов: расширения системы nbgrader с поддержкой автоматической проверки открытых ответов через БЯМ и системы проверки итогового проекта курса.

Первый компонент представляет собой расширение стандартного функционала nbgrader. В текущем виде инструмент поддерживает два типа ячеек: автоматически проверяемые ячейки с кодом на основе assert-тестов и ячейки с открытым ответом, проверяемые вручную. Предлагается ввести третий тип — ячейки с открытым ответом, оцениваемые автоматически с привлечением БЯМ. Для каждой такой ячейки преподаватель указывает условие задания, эталонный ответ и критерии оценивания. При проверке модель получает эти данные вместе с ответом студента и возвращает числовую оценку и содержательную обратную связь. Такой подход позволяет автоматизировать оценку коротких теоретических вопросов, формулировок выводов и интерпретаций результатов — тех типов заданий, которые принципиально важны для глубокого понимания материала, но традиционно требуют ручной проверки. Интеграция выполняется в рамках существующего интерфейса nbgrader, что не требует от преподавателей изменения привычного рабочего процесса.

Второй компонент — система проверки итогового проекта. В отличие от стандартных заданий проект представляет собой развёрнутую самостоятельную работу студента: постановку задачи, анализ данных, обучение и сравнение моделей, интерпретацию результатов и выводы. Проект сдаётся с кодом и текстовыми пояснениями в открытом виде. БЯМ анализирует как программную, так и текстовую части работы: корректность реализации, качество анализа, обоснованность выводов и полноту изложения. По результатам анализа модель формирует развёрнутую персонализированную обратную связь по каждому разделу работы и дает

оценку. Данный подход позволяет предоставить студенту детальный разбор его работы без привлечения преподавателя для проверки каждого проекта индивидуально, а также расширить перечень решаемых задач за счёт нетиповых и открытых постановок.

Для апробации предложенных инструментов разрабатывается курс по классическому машинному обучению, охватывающий базовые разделы дисциплины: линейные модели, решающие деревья, ансамблевые методы и обучение без учителя. Структура курса предусматривает домашние задания в формате Jupyter Notebook с ячейками кода, открытыми теоретическими вопросами и итоговым проектом. Апробация проводится на студентах НИУ ВШЭ, что позволяет оценить работу системы в условиях реального учебного процесса.

ГЛАВА 2

2.1 Рабочая программа дисциплины

2.1.1 Пререквизиты

Для успешного освоения курса студент должен владеть:

– основами программирования на Python (синтаксис, функции, классы, работа с библиотеками);
– линейной алгеброй (матрицы, векторы, операции над ними);
– математическим анализом (производные, градиент);
– основами теории вероятностей и математической статистики.

2.1.2 Разделы

Курс состоит из восьми тематических разделов, охватывающих основные направления классического машинного обучения:

1) Введение в машинное обучение. Инструменты: NumPy, Pandas.
2) Линейная регрессия и методы оптимизации.
3) Классификация: логистическая регрессия, метод опорных векторов.
4) Деревья решений.
5) Ансамблевые методы: случайный лес, градиентный бустинг.
6) Кластеризация.
7) Специализированная тема по выбору: обработка естественного языка, компьютерное зрение или рекомендательные системы.
8) Консультация по проекту и защита проектов.

2.1.3 Темы

В рамках каждого раздела рассматриваются следующие темы:

– Разведочный анализ данных, предобработка, feature engineering, работа с дисбалансом классов.
– Математические основы машинного обучения: функции потерь, методы оптимизации (градиентный спуск и его модификации: SGD, Momentum, Adam).
– Линейные модели: линейная и логистическая регрессия, метод наименьших квадратов.
– Метрики качества классификации и регрессии: Precision, Recall, F1-мера,

ROC-AUC, PR-AUC, MSE, RMSE, MAE.
- Метод опорных векторов и ядровой трюк.
- Деревья решений для регрессии и классификации: критерии разбиения, переобучение, регуляризация.
- Ансамблевые методы: бэггинг, случайный лес, градиентный бустинг, bias-variance trade-off.
- Методы кластеризации и уменьшение размерности.

2.1.4 Оцениваемые задания

2.1.4.1 Домашние задания

На курсе предусмотрено три домашних задания, каждое рассчитано на две недели выполнения. Дедлайны строгие; работы, поданные после срока, не принимаются. Итоговая оценка за домашние задания — среднее арифметическое трёх работ.

- Домашнее задание 1. Работа с NumPy: векторизованные вычисления, операции над массивами.
- Домашнее задание 2. Градиентный спуск. Реализация с нуля классов VanillaGradientDescent, StochasticGradientDescent, MomentumDescent, Adam, а также класса LinearRegression с поддержкой L1/L2-регуляризации. Все реализации должны быть векторизованы. С использованием разработанных классов необходимо обучить модель, предсказывающую выбросы CO2. Датасет был самостоятельно сформирован из открытых источников. За прохождение определенных порогов целевой метрики студенты получали дополнительные баллы.
- Домашнее задание 3. Классификация текста. Предобработка текста, построение признаковых представлений, обучение и настройка классических моделей (логистическая регрессия, SVM), анализ метрик классификации и интерпретация результатов на задаче определения автора текста (человек или нейросеть). Домашнее задание частично выполняется в формате соревнования — наивысшую оценку (10 баллов) могут получить только топ-10 студентов, получивших максимальное значение целевой метрики на валидационной выборке.

2.1.4.2 Активность на семинарах

В ходе каждого семинара студентам предлагаются бонусные задания и ответы на вопросы. За одно занятие можно получить не более четырёх отметок активности. Итоговая оценка за активность — среднее за все занятия семестра. Шкала перевода:

– 4 отметки — 10 баллов;
– 3 отметки — 7,5 балла;
– 2 отметки — 5 баллов;
– 1 отметка — 2,5 балла;
– 0 отметок — 0 баллов.

Вопросы на семинарах формулировались так, чтобы подтолкнуть студентов к самостоятельному выводу, не давая ответ напрямую. Примеры вопросов: влияние коэффициента регуляризации λ на поведение модели при его уменьшении или росте; причины отказа от масштабирования OneHot-закодированных признаков; интерпретация метрики R^2.

Часть заданий носила практический характер: студентам предлагалось написать код для обучения модели или провести эксперименты с гиперпараметрами — например, исследовать влияние параметров случайного леса на качество модели и OOB-score.

2.1.4.3 Проект

Проект выполняется индивидуально (командные работы допускаются в исключительных случаях по согласованию с преподавателем). Работа ведётся в репозитории GitHub Classroom и сдаётся поэтапно через четыре чекпоинта с последующей защитой.

Требования к проекту:

– датасет — не менее 10 000 строк и не менее 9 признаков;
– наличие чётко сформулированной задачи машинного обучения;
– уникальность: датасет не должен пересекаться с проектами других студентов;
– при использовании Kaggle — только задачи типа monetary, задачи Getting Started не допускаются.

Структура сдачи и максимальные баллы:

– CP0 (выбор проекта, регистрация в GitHub Classroom) — допускной этап;
– CP1 (предобработка данных, базовое моделирование, качество кода) — 25 баллов, проходной порог 13 баллов;
– CP2 (расширенное моделирование, перебор гиперпараметров, ансамбли) — 30 баллов, проходной порог 15 баллов;
– CP3 (отчёт, деплой через FastAPI или аналог) — 30 баллов, проходной порог 15 баллов;
– защита (презентация 5 минут, ответы на вопросы) — 15 баллов.

Не набравший проходной балл на текущем чекпоинте не допускается к следующему; итоговая оценка за проект формируется из уже накопленных баллов.

2.1.5 Результаты обучения

По итогам курса студент:

– знает математические основы классических алгоритмов машинного обучения: линейные модели, деревья решений, ансамблевые методы, методы кластеризации;
– умеет реализовывать алгоритмы оптимизации (градиентный спуск и его модификации) с нуля на Python с использованием NumPy;
– умеет проводить полный цикл работы с данными: разведочный анализ, предобработку, feature engineering, построение и отбор моделей;
– умеет выбирать и обосновывать метрики качества для задач регрессии и классификации;
– имеет практические навыки работы с текстовыми данными и признаковыми представлениями;
– умеет организовывать воспроизводимый ML-проект: структура кода, линтеры, фиксация зависимостей;

2.2 Внедрение автоматизированной проверки в nbgrader

В стандартной версии nbgrader для ячеек с кодом доступны следующие типы: Autograded answer - ячейка, в которой студент пишет код, проверяемый автоматическими assert-тестами; Autograded tests - ячейка с тестами, скрытая от студента и запускаемая при проверке; Manually graded answer - ячейка с кодом или текстом, требующая ручной оценки; Manually graded task - аналогичный тип, но оценка выставляется на уровне задания целиком; Read-

only - ячейка, недоступная для редактирования студентом. Для текстовых (Markdown) ячеек набор типов ограничен: Manually graded answer, Manually graded task и Read-only. Таким образом, автоматическая проверка предусмотрена только для кода, а текстовые ответы могут быть оценены исключительно вручную.

2.2.1 Требования к работе

На основании проведенного анализа были сформулированы следующие требования к разрабатываемому решению.

Требования к новому типу ячейки:

1) В nbgrader должен быть добавлен новый тип ячейки - LLM graded answer, доступный для выбора наряду со стандартными типами.
2) Ячейка должна поддерживать структурированную разметку с разделителями: блок с формулировкой задания (видимый студенту) и блок с эталонным ответом и критериями оценивания (скрытый от студента).
3) Разделители должны быть настраиваемыми через конфигурацию.

Требования к генерации студенческой версии:

1) При генерации студенческой версии задания блок с критериями должен автоматически удаляться и заменяться текстом-заглушкой.
2) Формулировка задания должна сохраняться в неизменном виде.
3) Генерация должна выполняться стандартной командой nbgrader без дополнительных утилит.

Требования к конфигурации и промпту:

1) Преподаватель должен иметь возможность задать параметры подключения к API языковой модели: ключ, адрес, название модели.
2) Должен поддерживаться настраиваемый шаблон промпта с подстановкой переменных: формулировка задания, ответ студента, вывод ячейки, критерии оценивания, максимальный балл.
3) Должен быть задан таймаут ожидания ответа от API.

Требования к выходу модели и интеграции:

1) Модель должна возвращать численную оценку в диапазоне от 0 до максимального балла за задание.
2) Проверка ячеек типа LLM graded answer должна запускаться стандартной командой nbgrader autograde совместно с проверкой остальных типов

ячеек.

3) Результаты проверки должны сохраняться в базу данных nbgrader, обеспечивая совместимость с существующим интерфейсом просмотра оценок.

2.2.2 Реализация расширения nbgrader

В рамках работы в инструмент nbgrader был добавлен новый тип ячейки - LLM graded answer, предназначенный для заданий, ответ на которые требует проверки языковой моделью. На рис. 2.1 показан пример выбора типа ячейки с добавленным вариантом.

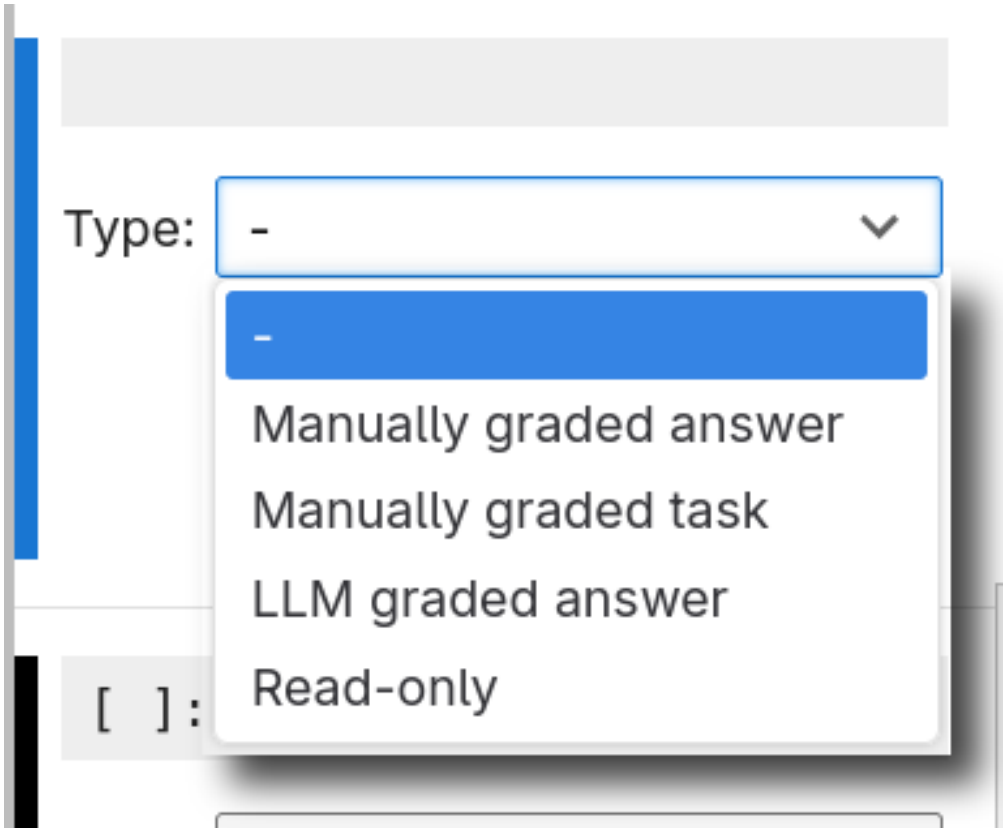

Рисунок 2.1 — Типы ячеек в nbgrader

2.2.3 Создание задания преподавателем

При подготовке преподавательской версии задания в Jupyter Notebook преподаватель выбирает тип ячейки LLM graded answer и заполняет её с помощью специальных разделителей, принятых в nbgrader:

– блок BEGIN QUESTION_LLM ... END QUESTION_LLM содержит формулировку задания, видимую студенту;
– блок BEGIN CRITERIA_LLM ... END CRITERIA_LLM содержит эталонный ответ и критерии оценивания, скрытые от студента.

Пример разметки ячейки приведён на рис. 2.2.

Задание 2.1. Градиент MSE в матричном виде (0.04 балла).

BEGIN QUESTION_LLM

Напомним, что функция потерь MSE записывается как:

$$Q(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \langle x_i, w\rangle)^2 = \frac{1}{n}\|Xw - y\|^2$$

где n – количество объектов в выборке, $X \in \mathbb{R}^{n\times d}$ – матрица "объект-признак", а $y \in \mathbb{R}^n$ – целевая переменная. Через x_i обозначается i-ая строчка матрицы X, отвечающая за i-й объект выборки.

Выпишите ниже (подсмотрев в семинар или решив самостоятельно) градиент для функции потерь MSE в матричном виде.

END QUESTION_LLM

BEGIN CRITERIA_LLM

Эталонный ответ: Градиент MSE в матричном виде: $\|Xw - y\|^2$ по w: $2X^\top(Xw - y)/n$.

Критерии оценивания (4 балла):

- 4 балла: указана правильная формула $2X^\top(Xw - y)/n$ в различных записях (latex, markdown)
- 0 баллов: неверная формула или отсутствие ответа.

END CRITERIA_LLM

Рисунок 2.2 — Пример ячейки, заполненной преподавателем

2.2.4 Генерация студенческой версии

Генерация студенческой версии задания выполняется стандартной командой nbgrader: nbgrader generate_assignment.

В результате блок с критериями удаляется, а на его месте появляется текст YOUR ANSWER HERE, сохраняя при этом формулировку задания в неизменном виде. Пример отображен на рис. 2.3.

Задание 2.1. Градиент MSE в матричном виде (0.04 балла). ¶

Напомним, что функция потерь MSE записывается как:

$$Q(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \langle x_i, w\rangle)^2 = \frac{1}{n}\|Xw - y\|^2$$

где n – количество объектов в выборке, $X \in \mathbb{R}^{n\times d}$ – матрица "объект-признак", а $y \in \mathbb{R}^n$ – целевая переменная. Через x_i обозначается i-ая строчка матрицы X, отвечающая за i-й объект выборки.

Выпишите ниже (подсмотрев в семинар или решив самостоятельно) градиент для функции потерь MSE в матричном виде.

YOUR ANSWER HERE

Рисунок 2.3 — Пример ячейки, которую видит студент

2.2.5 Конфигурация и запуск проверки

Перед запуском автоматической проверки преподаватель заполняет конфигурационный файл, задающий параметры обращения к БЯМ. Ключевые поля конфигурации:

- llm_api_key, llm_api_base, llm_model - параметры подключения к API языковой модели;
- БЯМ_prompt_template - шаблон промпта, в который подставляются:
  - {question} - формулировка задания;
  - {student_answer} - ответ студента;
  - {cell_output} - вывод ячейки (если есть);
  - {grading_instructions} - критерии оценивания;
  - {max_points} - максимальный балл за задание.
- begin_question_delimiter, end_question_delimiter и аналогичные поля для критериев - настройка разделителей разметки; по умолчанию равны BEGIN QUESTION_LLM и END QUESTION_LLM для заданий и BEGIN CRITERIA_LLM и END CRITERIA_LLM для критериев.
- llm_timeout - максимальное время ожидания ответа от API (в секундах).

Модель получает на вход перечисленные данные и возвращает число от 0 до max_points, представляющее выставленный балл.

После получения ответов студентов преподаватель запускает проверку стандартной командой: nbgrader autograde.

Система автоматически обходит все ячейки типа LLM graded answer, отправляет соответствующие запросы к языковой модели и сохраняет выставленные баллы в базу данных nbgrader. Остальные типы ячеек проверяются штатными механизмами инструмента.

2.3 Создание системы для проверки проектов

ГЛАВА 3

ЗАКЛЮЧЕНИЕ

В заключении подведены итоги проведённого исследования. Разработанная методика применения алгоритмов глубокого обучения для классификации данных показала свою эффективность на тестовых наборах. Проведённый анализ литературы и экспериментальные исследования подтвердили, что предложенный подход позволяет снизить процент ошибок классификации по сравнению с традиционными методами.

Основные выводы работы:

– разработанная методика оправдана с точки зрения теоретических обоснований и практического применения;
– экспериментальные результаты демонстрируют преимущества глубоких нейронных сетей для задач классификации;
– работа содержит рекомендации для дальнейших исследований и внедрения метода в информационные системы.

Таким образом, проведённое исследование подтверждает перспективность использования методов глубокого обучения в решении актуальных задач обработки данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Соколов Е. А. Машинное обучение 1. Курс факультета компьютерных наук НИУ ВШЭ. — дата обращения: 14.04.2026. URL: https://github.com/esokolov/ml-course-hse.
2. Курс машинного обучения факультета ВМК МГУ. — дата обращения: 14.04.2026. URL: https://github.com/MSU-ML-COURSE.
3. Оценивание курса ВМК МГУ. — дата обращения: 15.04.2026. URL: https://github.com/MSU-ML-COURSE/ML-COURSE-25-26/blob/master/slides/intro/ML$_g rading_s ystem.pdf$.
4. Открытый курс машинного обучения mlcourse.ai от Open Data Science. — дата обращения: 15.04.2026. URL: https://ods.ai/tracks/classic-models-spring26.
5. Deep Learning School при ФПМИ МФТИ. — дата обращения: 14.04.2026. URL: https://dls.samcs.ru/.
6. Открытый курс машинного обучения girafe-ai. — дата обращения: 14.04.2026. URL: https://github.com/girafe-ai/ml-course.
7. Karpov.Courses: Инженер машинного обучения. — дата обращения: 14.04.2026. URL: https://karpov.courses/ml-start.
8. nbgrader: A system for assigning and grading notebooks. — дата обращения: 14.04.2026. URL: https://nbgrader.readthedocs.io/en/stable/.
9. nbgrader: Github repository. — дата обращения: 14.04.2026. URL: https://github.com/jupyter/nbgrader.
10. Otter-Grader: A Python and R autograding solution. — дата обращения: 14.04.2026. URL: https://otter-grader.readthedocs.io/en/latest/.
11. Otter-Grader: Github repository. — дата обращения: 14.04.2026. URL: https://github.com/ucbds-infra/otter-grader.
12. CodeGrade: The Engaging Code Learning Platform. — дата обращения: 14.04.2026. URL: https://www.codegrade.com/.

13. PyEvalAI: AI-assisted evaluation of Jupyter Notebooks for immediate personalized feedback. — дата обращения: 14.04.2026. URL: https://arxiv.org/abs/2502.18425.
14. Яндекс Контест — платформа для решения задач по программированию. — дата обращения: 14.04.2026. URL: https://contest.yandex.ru/.
15. Stepik: образовательная онлайн-платформа. — дата обращения: 14.04.2026. URL: https://stepik.org/.
16. Use autograding — GitHub Classroom Documentation. — дата обращения: 15.04.2026. URL: https://docs.github.com/en/education/manage-coursework-with-github-classroom/teach-with-github-classroom/use-autograding.
17. Kaggle Competitions. — дата обращения: 15.04.2026. URL: https://www.kaggle.com/competitions.

ПРИЛОЖЕНИЕ. А РУКОВОДСТВО ПО ИСПОЛЬЗОВАНИЮ ШАБЛОНА

В данном приложении приведены примеры оформления основных элементов ВКР согласно требованиям ИТМО (версия 4.0, 2022 г.).

Пример заголовков разделов и подразделов

Ниже показаны три допустимых варианта оформления заголовков (п. 4.4.4 требований ИТМО).

Вариант 1	**Вариант 2**	**Вариант 3**
1 НАЗВАНИЕ РАЗДЕЛА **1.1 Название подраздела** 1.1.1 Название подпункта 1.1.2 Название подпункта	**1 Название раздела** **1.1 Название подраздела** 1.1.1 Название подпункта 1.1.2 Название подпункта	1 Название раздела 1.1 Название подраздела 1.1.1 Название подпункта 1.1.2 Название подпункта

Правила оформления заголовков:

– заголовки разделов и подразделов начинаются с абзацного отступа,
– в конце заголовка точка не ставится,
– переносы слов в заголовках не допускаются,
– аббревиатуры в заголовках не допускаются.

Пример изображения

Ссылка на рисунок обязательна в тексте перед его вставкой (рисунок А.1):

Примеры таблиц

Ссылка на таблицу обязательна перед её вставкой (таблица А.1):

Таблица А.1 — Пример построения таблицы

Заголовок	**Заголовок колонки**		**Заголовок колонки**
	Подзаголовок	**Подзаголовок**	
A	B	C	D

Пример таблицы с переносом на следующую страницу (таблица А.2):

Рисунок А.1 — Пример изображения, вставленного в приложение

Таблица А.2 — Пример переноса таблицы

Заголовок	**Заголовок колонки**	**Заголовок колонки**	**Заголовок колонки**
1	2	3	4
2	3	4	5

Продолжение таблицы А.2

Заголовок	**Заголовок колонки**	**Заголовок колонки**	**Заголовок колонки**
3	4	5	6
4	5	6	7

Примеры перечислений

Вариант 1 — маркированный список через тире (п. 4, Приложение 1 требований ИТМО):

Информационно-сервисная служба для обслуживания удалённых пользователей включает следующие модули:

– удалённый заказ,
– виртуальная справочная служба,

– виртуальный читальный зал.

Вариант 2 — нумерованный список с цифрой и скобкой:

Работа по оцифровке включала следующие технологические этапы:

1) первичный осмотр и структурирование исходных материалов,
2) сканирование документов,
3) обработка и проверка полученных образов,
4) выходной контроль качества.

Вариант 3 — многоуровневый список:

Разрабатываемое устройство можно применять в различных отраслях:

– в машиностроении:
  1) для очистки отливок от формовочной смеси,
  2) для холодной штамповки из листа;
– в ремонте техники:
  1) устранение наслоений на внутренних стенках труб,
  2) очистка каналов небольшого диаметра.

Пример формулы

Функция потерь определяется формулой (А.1):

$$L(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \tag{А.1}$$

где y_i — истинное значение, $\hat{y}_i$ — предсказанное значение, n — объём выборки.

Инструкция: Приложение содержит дополнительные материалы, не вошедшие в основной текст работы. В него можно включать изображения, таблицы, программный код, результаты экспериментов и другие вспомогательные данные.